

File I/O and Strings

CSC 230 : C and Software Tools

NC State Department of Computer
Science

These slides will be updated for
fall 2026.

Topics for Today

- File I/O
- Text Files
- Binary Files
- Pattern matching with scanf

File I/O

- Our good friend, `stdio.h`, also includes support for file I/O

```
#include <stdio.h>
```

- We just need to ask it to open a new file.

```
FILE *stream = fopen( "file-name.txt", "r" );
```

Path to the file you want.

Mode String (reading, writing, etc).

- This creates a new *stream* and returns a *file pointer* for working with it.
 - Or it returns **NULL on failure**.

File Mode

- When you open a file, the *mode string* says how you want to access it.

```
FILE *fp = fopen( "file-name.txt", "r" );
```



Mode String	Access Type
"r"	Open for reading
"w"	Open for writing (created and truncated)

More mode strings later.

Formatted I/O

- We have a version of `printf( )` for writing to any stream.

```
int fprintf(FILE *stream, const char format[], ...);
```

Stream you want
to write to.

The rest is just like
`printf`.

- We have a similar version of `scanf( )`

```
int fscanf(FILE *stream, const char format[], ...);
```

Same here, just one new
parameter at the start.

Finishing I/O

- Streams perform *buffering*
 - They keep I/O characters in memory to consolidate costly system calls to the OS
- You need to tell the system when you're done with a file.

```
int fclose( FILE *stream );
```

- This will flush any buffered data.
- Forgetting to close a file is called a *file leak*

Leaking Files

- There's typically a limit on the number of open files you can have
 - That's why leaking files is bad
- I have a test for this: `openTest.c`
 - It opens files over and over until it can't create any more
 - Let's give it a try

Formatted Input Example

- An example, reading a list of numbers:

```
FILE *fp = fopen( "numbers.txt", "r" );  
if ( !fp ) {  
    ...  
}  
  
int total = 0;  
  
int val;  
while ( fscanf( fp, "%d", &val ) == 1 )  
    total += val;  
  
fclose( fp );  
  
printf( "Total: %d\n", total );
```

I want to open for reading.

I want to read from a file,
instead of standard input.

I'm all done with this file.

Formatted Output Example

- An example, writing a file of numbers.

```
FILE *fp = fopen( "output.txt", "w" );  
if ( !fp ) {  
    ...;  
}  
  
for ( int i = 0; i < 25; i++ )  
    fprintf( fp, "%f\n", i * i * 3.141592653589793 );  
  
fclose( fp );
```

I want to open for writing.

I want to write to a file
instead of standard output.

All done, flush any buffered
output.

Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.

What you see in a text editor.

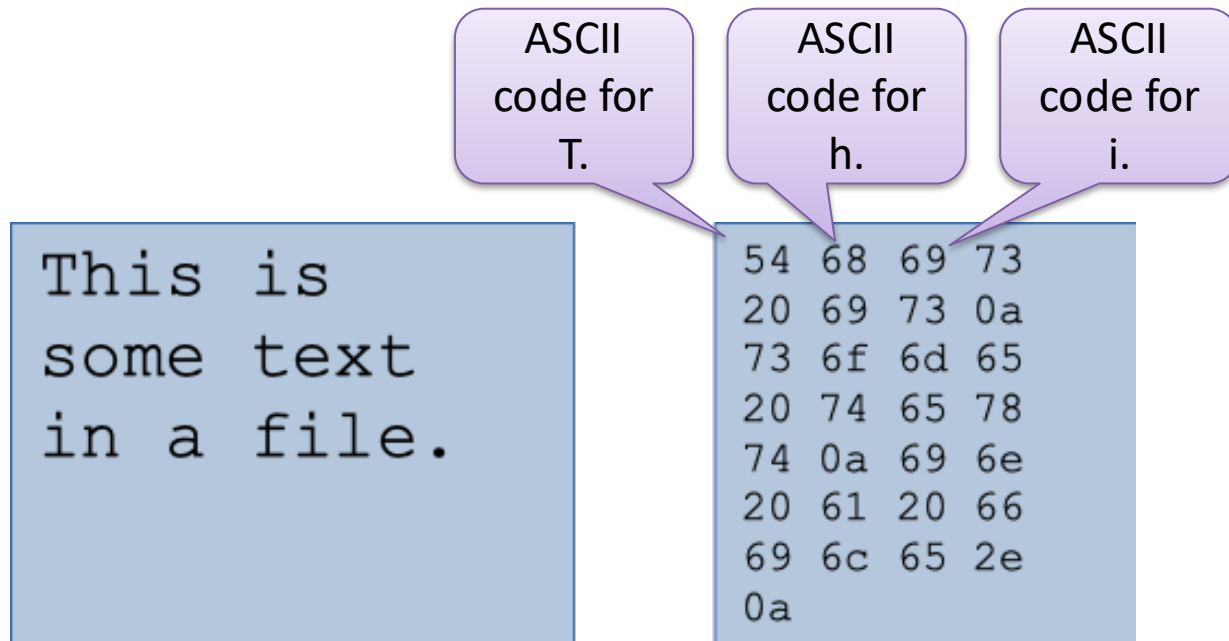
```
This is  
some text  
in a file.
```

What's really in the file.

```
54 68 69 73  
20 69 73 0a  
73 6f 6d 65  
20 74 65 78  
74 0a 69 6e  
20 61 20 66  
69 6c 65 2e  
0a
```

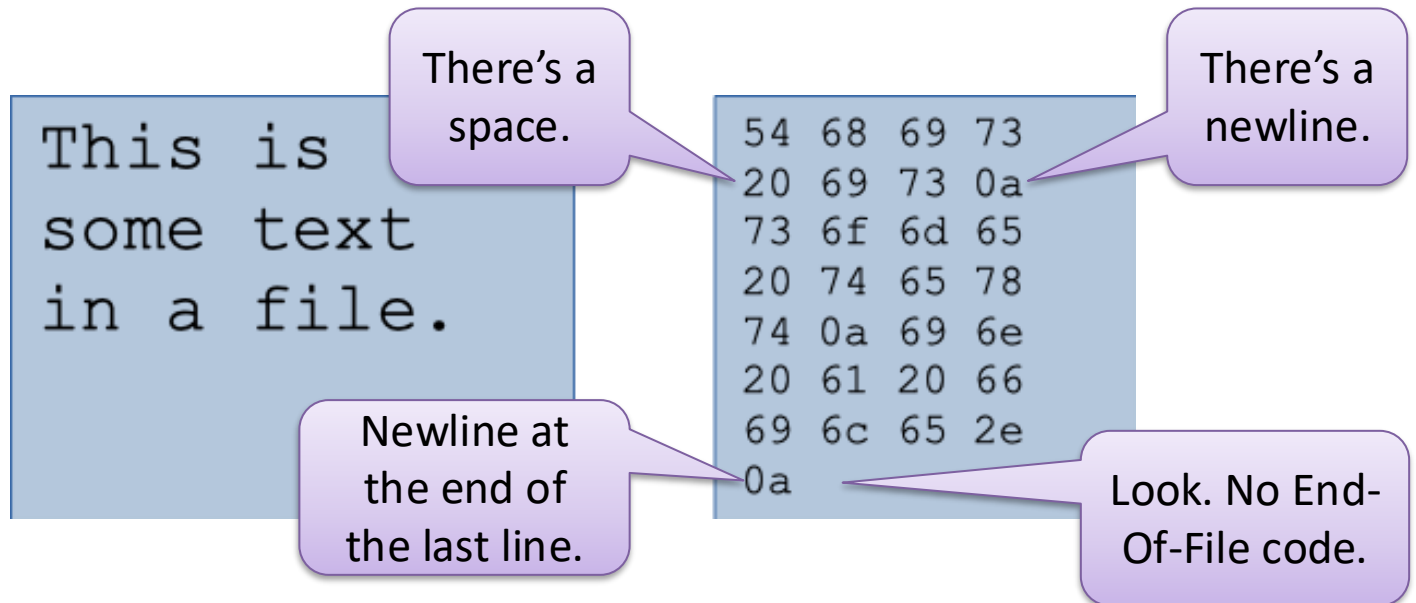
Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.



Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.



Meet Hexdump

- The hexdump command can show you the exact sequence of bytes in a file.
 - It has options to show file contents in various formats.

```
$ hexdump -C someFile.txt
```

Show numeric codes
and characters.

File you want to see.

Looking at Source Code

- A source file is like any other text file.
- Hexdump can show you exactly what's inside.

hello.c

```
#include <stdio.h>

int main()
{
    printf( "Hello World\n" );
    return 0;
}
```

```
$ hexdump -C hello.c
```

00000000	23 69 6e 63 6c 75 64 65	20 3c 73 74 64 69 6f 2e	#include <stdio.
00000010	68 3e 0a 0a 69 6e 74 20	6d 61 69 6e 28 29 0a 7b	h>..int main().{
00000020	0a 20 20 70 72 69 6e 74	66 28 20 22 48 65 6c 6c	. printf("Hell
00000030	6f 20 57 6f 72 6c 64 5c	6e 22 20 29 3b 0a 20 20	o World\n");.
00000040	72 65 74 75 72 6e 20 30	3b 0a 7d 0a	return 0;}.

Bad Text Formatting

- Here's an example that breaks some style guide rules.

hello_bad.c

```
#include <stdio.h>

int main()
{
    printf( "Hello World\n" );
    return 0;
}
```

There's a CR LF
for line
termination.

and there's a hard
tab.

```
$ hexdump -C hello_bad.c
00000000  23 69 6e 63 6c 74 64 65 20 3c 73 74 64 69 6f 2e |#include <stdio.|
00000010  68 3e 0d 0a 0d 0a 69 6e 74 20 6d 61 69 6e 28 29 |h>...int main()|
00000020  0d 0a 7b 0d 0a 09 70 72 69 6e 74 66 28 20 22 48 |...{...printf( "H|
00000030  65 6c 6c 6f 20 57 6f 72 6c 64 5c 6e 22 20 29 3b |ello World\n" );|
00000040  0d 0a 09 72 65 74 75 72 6e 20 30 3b 0d 0a 7d |...return 0;...}|
```

No line termination at
the end of the last
line.

Wide Characters

- One-byte codes aren't sufficient for all character sets.
- C supports a wide character type, **wchar_t**
- In a file, a format like UTF-8 supports unicode
 - Using 1-byte codes for ASCII characters
 - ... and multi-byte codes for other characters.

```
$ hexdump -C uni.txt
00000000  61 62 63 ce 91 ce 92 ce 93 0a      |abc.....|
```

One-byte codes
for a, b, c

Two-byte codes
for Greek Α, Β, Γ

Character I/O

- We have functions for character I/O on the terminal

```
int getchar(void) ;
```

```
int putchar(int ch) ;
```

- They read/write a single character
- Return EOF on failure.

- We have similar functions for working with any stream

```
int fgetc(FILE *stream) ;
```

I read a single char
from the given stream.

```
int fputc(int ch, FILE *stream) ;
```

I write a single char to
the given stream.

Character I/O Example

- Copying a file character-by-character:

```
FILE *src = fopen( "source.txt", "r" );  
FILE *dest = fopen( "dest.txt", "w" );  
  
int ch;  
while ( ( ch = fgetc( src ) ) != EOF )  
    fputc( ch, dest );  
  
fclose( src );  
fclose( dest );
```

Open a source and destination file.

Copy characters until we hit EOF.

Clean up when we're done.

Take Backsies

- An input stream will let you return **one input character** to the buffer

```
int ungetc(int c, FILE *stream);
```

- Pushes character `c` back onto the given stream
- Clears the EOF condition (if you've already hit EOF)
- Oh, and you can't push back an EOF, just a real character.
- It even works with **stdin**

Taking Another Look

- Normally, files are accessed **sequentially**
 - After you read/write a byte you automatically get the next byte
 - Buffering done inside the library promotes sequential access performance.
- But, you you can go back to the start of a file.
 - To, re-read or re-write it from the start.

```
int rewind( FILE *stream );
```

Jumping Around

- There's a more general function to seek to anywhere.

What stream?

Where to?

With respect to what?

```
int fseek(FILE *stream, long offset, int whence);
```

- The whence parameter tells you with-respect to what?
 - SEEK_SET : just go to this offset
 - SEEK_CUR : offset is with-respect-to the current position
 - SEEK_END : offset is from the end of the file.
- Works for binary streams, limited functionality for text

```
fseek( stream, -1, SEEK_CUR ); // Backup one byte
fseek( stream, 0, SEEK_END ); // Goto end of file
fseek( stream, p, SEEK_SET ); // Goto position p
```

Checking Your Location

- How do you know where to seek to in a file?
- You could count characters as you read.
 - But, that sounds like some extra work.
 - And, it may not work very well for text files.
 - There's some conversion that occurs as you read a text file (e.g., `\r\n` $\rightarrow$ `\n`) ... more about this later.
- But, a stream can tell you your current location.

```
long ftell( FILE *stream );
```

- You can `fseek` back to this location to return and read from there again.

Three Streams for Free

- Your program gets three streams at startup, you've been working with them all semester
 - **stdin** : standard input (from the terminal by default)
 - **stdout** : standard output (to the terminal by default)
 - **stderr** : standard error (also to the terminal by default)
- These are all global variables declared in **stdio.h**
- This is secretly why openTest.c could open 1021 files.
 - 3 files open at startup + 1021 more files opened = 1024 files.

Using the Standard Streams

- With names for our standard streams
 - This gives us alternative ways to do terminal I/O.

```
fscanf( stdin, ... );
```



```
scanf( ... );
```

```
fprintf( stdout, ... );
```



```
printf( ... );
```

```
fprintf( stderr, ... );
```

Maybe the first two aren't very useful, but here's something new.

- Printing to stderr is a standard way of reporting, well, errors.
 - Why? We can redirect it differently from stdout.

Using stderr

- We can use stderr for any error reports

```
FILE *fp = fopen( "numbers.txt", "r" );
if ( !fp ) {
    fprintf( stderr, "Can't open input file\n" );
    exit( 1 );
}

int len = 0;
if ( fscanf( input, "%d", &len ) != 1 ) {
    fprintf( stderr, "Can't read input length\n" );
    exit( 1 );
}

. . .
```

Redirecting Streams

- From the shell, we can choose to redirect streams differently.

I redirect standard output.

I redirect standard error.

```
$ ./program > output.txt 2> error.txt
```

- Or the same

Send standard error to the same file as standard output.

```
$ ./program > everything.txt 2>&1
```

- This syntax is dependent on your shell, but this is what bash uses.

Flushing the Buffer

- Streams perform buffering, but sometimes you want your output **right now!**

```
int fflush( FILE *stream );
```

- Buffered output is written out immediately
- Normally, standard output flushes ... when it sees a newline

```
printf( "You can't see me " );  
sleep( 5 );  
printf( "... until you write a newline\n" );
```

I see this mistake all the time
with debug output.

Flushing the Buffer

- You can take control of this with `fflush()`

```
printf( "But flushing writes output immediately" );  
fflush( stdout );  
sleep( 5 );  
  
printf( "... even without a newline.\n" );
```

Checking for EOF

- We have a simple, general technique for detecting the end of input.

```
while ( scanf( "%d", &v ) == 1 ) ...
```

- Call a reading function ... see if it returns successfully.
 - Remember when you hit EOF and stop reading.
- LibC remembers this for us automatically.

```
int feof(FILE *stream);
```

- Returns true if you've already reached the EOF.
 - Set whenever a reading function encounters EOF (even internally).

Easy Mistakes With EOF

- This gives us a new, terrible way to try to detect the end of input.

```
while ( ! feof( input ) ) {  
    fscanf( input, "%d", &value );  
    printf( "got: %d\n", value );  
}
```

This is bad.
Don't do this.

25 102**EOF**

EOF right after the
second number.

output

25
102

25 102\n**EOF**

Some whitespace
before the EOF.

output

25
102
102

Know What You're Doing

- `feof()` works exactly as advertised
- ... but you have to think about what's going on internally.

123EOF

Both of these will read right up to here.

```
int ch1 = getchar();  
int ch2 = getchar();  
int ch3 = getchar();  
if ( feof( stdin ) )  
    printf( "At EOF\n" );
```

I won't report EOF yet.

```
int val;  
scanf( "%d", &val );  
if ( feof( stdin ) )  
    printf( "At EOF\n" );
```

But I will.

Things go Wrong

- A file can encounter an error condition.
 - If you try to write to a file that's open for reading.
 - Or the other way around
 - Or the disk is full
 - Or you reach the maximum files size.
- When this happens, the stream notices and remembers it.
 - If you know how to ask, it will tell you something already went wrong:

```
int ferror(FILE *stream);
```

Forgetting Past Mistakes

- The EOF and error conditions remain set
- ... until you explicitly clear them.
- There's a function for that:

```
void clearerr(FILE *stream);
```

String Parsing Options

- The %s format specifier reads
 - space-delimited words ... of arbitrary length.

```
char str[ 11 ];  
scanf( "%s", str );
```

Potential buffer overflow

- You can prevent buffer overflow by providing a field width.

```
char str[ 11 ];  
scanf( "%10s", str );
```

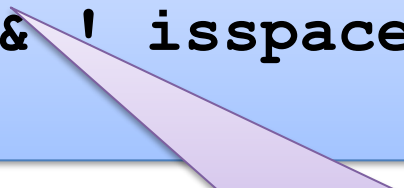
Read at most a 10-character word.

- How can you tell if the input word was too long?

Detecting Excessive Input

- Option 1 : check the next character.
 - If it's still a word character, scanf must have stopped reading inside a word.

```
char str[ 11 ];  
if ( scanf( "%10s", str ) == 1 ) {  
    int ch = getchar();  
    if ( ch != EOF && ! isspace( ch )  
        // bad input
```



Peek ahead at the next character.

Detecting Excessive Input

- Option 2 : permit one extra character in the input.
 - This is enough to let you detect when the input was too long.
 - (without actually reading the whole thing)

```
char str[ 12 ];  
if ( scanf( "%11s", str ) == 1 ) {  
    if ( strlen( str ) > 10 )  
        // bad input
```

One extra character.

Was the input too long?

String Parsing Options

- The %s format specifier reads space-delimited words.
- scanf() also supports a subset of regular expression syntax.
 - Giving you more control of the string you read.
 - Using square brackets we can define a *character class*, a set of exactly what characters to match.

```
scanf( "[%abc]", str );
```

Match a sequence of one or more a, b and c characters.

Doesn't skip spaces.

Character Class Examples

```
scanf( "[%abc]", str );
```

bcabbxaccb

This will return zero matches.

zero

```
scanf( "[% 0123456789]", str );
```

123 45w xyz 789

Not (Quite) Regular Expressions

- Regular expressions : a common pattern language for matching strings
- `scanf()/fscanf()` support a similar syntax
 - but, it's not exactly the same
 - generally not as powerful / expressive

String Parsing Options

- Inside the brackets, we can specify ranges of characters.

```
scanf( "%[A-Z]", str );
```

Match any capital letter.

- We can combine ranges and individual characters.

```
scanf( "%[a-zA-Z.]", str );
```

Match lower-case or capital letters or dot.

abc.XYZ.123

String Parsing Options

- You can negate a character class by starting it with a [^]

```
scanf( "%[^\\n]", str );
```

Match anything but a newline.

```
scanf( "%[^a-zA-Z]", str );
```

Anything but a letter.

123-456-abcd

String Parsing Options

- Using `*`, we can tell `scanf` to match a format specifier, but not save it:

```
scanf( "%*s%s", str );
```

Skip spaces, then save the next word in `str`.

Match a word (but don't store it)

- Really, this works with any format specifier.

```
scanf( "%*d%d", &val );
```

```
scanf( "%*[a-z]%20[A-Z]", str );
```

Skipping Whitespace

- Any whitespace in a format string ... matches any amount of whitespace.

```
scanf( "%c", &ch );
```

I'll read the next character, even if it's whitespace

```
scanf( " %c", &ch );
```

I'll skip all whitespace, then read a character

```
scanf( "[%0-9]", str );
```

I'll read just digits.

```
scanf( " [%0-9]", str );
```

I'll skip whitespace, then read digits.

Skipping Whitespace

- Be careful, stray whitespace can give seemingly strange behavior.

```
scanf( "%d%d\n", &a, &b );
```

OK. You want me to skip whitespace after reading these ints.

I better not return until I'm sure I have it all.

Matching Literal Characters

- Literal characters will match copies of themselves.

```
scanf( "%d+%d", &a, &b );
```

I'll match the +, but I won't skip spaces before it.

```
25+72
```

```
25 + 72
```

```
scanf( "%d +%d", &a, &b );
```

But I will.

```
scanf( "%d %%", &a );
```

Need to match a literal %? We got that.

Formatted Input from a String

- The standard library as functions to parse input from a file.
- There's also a function for parsing the contents of a string

```
int sscanf(const char str[], char const format[], ...);
```

- Parse the contents of str according to the given format string.
 - Assign remaining arguments based on the given conversion specifiers
 - Return the number of conversion specifiers matched.
- Kind of like wrapping a Scanner around a string in Java.
- These are declared in stdio.h

Formatted Input from a String

- Example, parse an int and a float from the contents of a buffer:

```
char buffer[] = "25 31.2";  
int iVar;  
float fVar;  
sscanf( buffer, "%d%f", &iVar, &fVar );
```

- This function is why I almost never use atoi(), atof() ...
 - sscanf() can do the job of all of them.
 - And it gives you a more useful success/failure report.

```
int val = atoi( str );
```

I return zero on an error.

```
if ( sscanf(str, "%d", &val) != 1 )  
    oops;
```

I save a value, and return a success/value status.

Checking your Progress

- The %n conversion specification tells sscanf() to report how many characters it has read so far.

```
sscanf( str, "%d +%n%d", &a, &n, &b );
```

25+72

317 + 214

Put the report here.

- The %n isn't included in the return value count.

Returns EOF, 0, 1 or 2

Formatted Output to a String

- We have similar functions for going the other way:

```
int sprintf(char str[], char const format[], ...);
```

```
int snprintf(char str[], size_t n, char const format[], ...);
```

- Like printf() but with output going to the given string.
- snprintf() writes at most n bytes (including a null terminator at the end)

There's a lot of inconsistency here.

- Example:

```
char buffer[ 10 ];  
snprintf( buffer, sizeof(buffer), "%s", "abcdefghijklmnop" );
```

Copies just the first 9 characters.